



Week 6

Hash Tables

Binary Search Trees

Algorithms and Data Structures
COMP3506/7505

Week 6 – Hash tables & search trees

1. Hash tables
 2. Binary search trees
 3. AVL trees (week 7)
- 

Week 6 – Hash tables & search trees

1. Hash tables
2. Binary search trees
3. AVL trees

Recall Map ADT

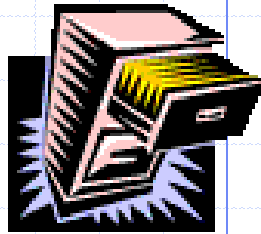
- Searchable collection of key-value entries
- Main operations
 - searching, inserting and deleting items
- Keys are unique
 - **cannot** have multiple entries with same key
- Can achieve $O(n)$ time complexity on basic operations using a list-based implementation

What We Want

- $O(1)$ worst case insert, find and delete

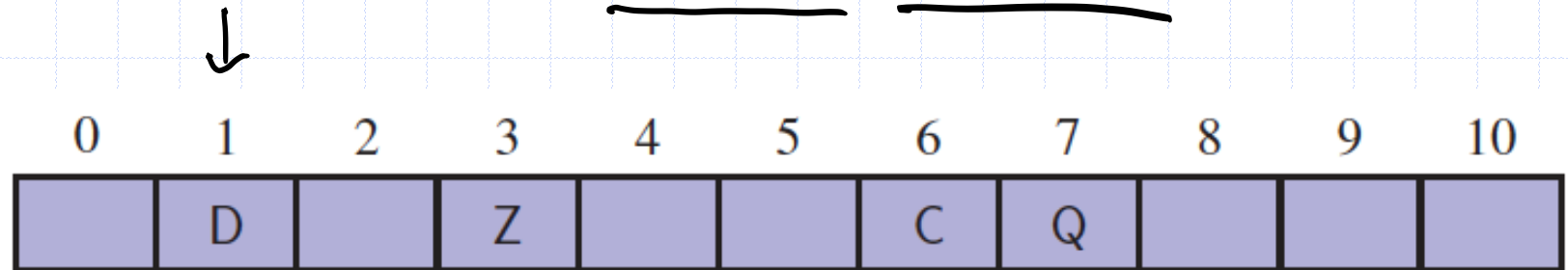
What We Can Have

- Require efficient insertion and find operations
- □ Do **not** require the items to be ordered
- □ Do **not** require efficient removal of items
- Ideally both insertion and find operations are $O(1)$ in the worst case
- Settle for $O(1)$ in the **average** case



Direct Access Array

- Consider a setting in which a map with n items uses keys that are known to be integers in a range from 0 to $N - 1$
 - for some $N \geq n$
 - Very good if key space is small (constant time insert/delete/lookup) – data is at position k
 - But if $N \gg n$ we will pay extreme space overhead
 - ♦ The array will be very sparse and wasteful

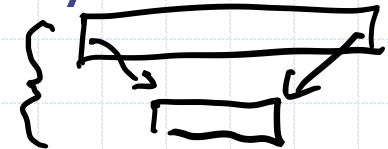


$$\text{mod}[\underbrace{\text{"hello"}}_{}] = 3$$

More General Kinds of Keys

- What should we do if our keys are not integers in the range from 0 to $N - 1$?

- use a **hash function** to map general keys to corresponding indices in a table



- Hash function is a mapping:

$$\{ \rightarrow h(k) : \{0, 1, \dots, U-1\} \rightarrow \{0, 1, \dots, N-1\}$$

- If $U \gg N$, no hash function is injective

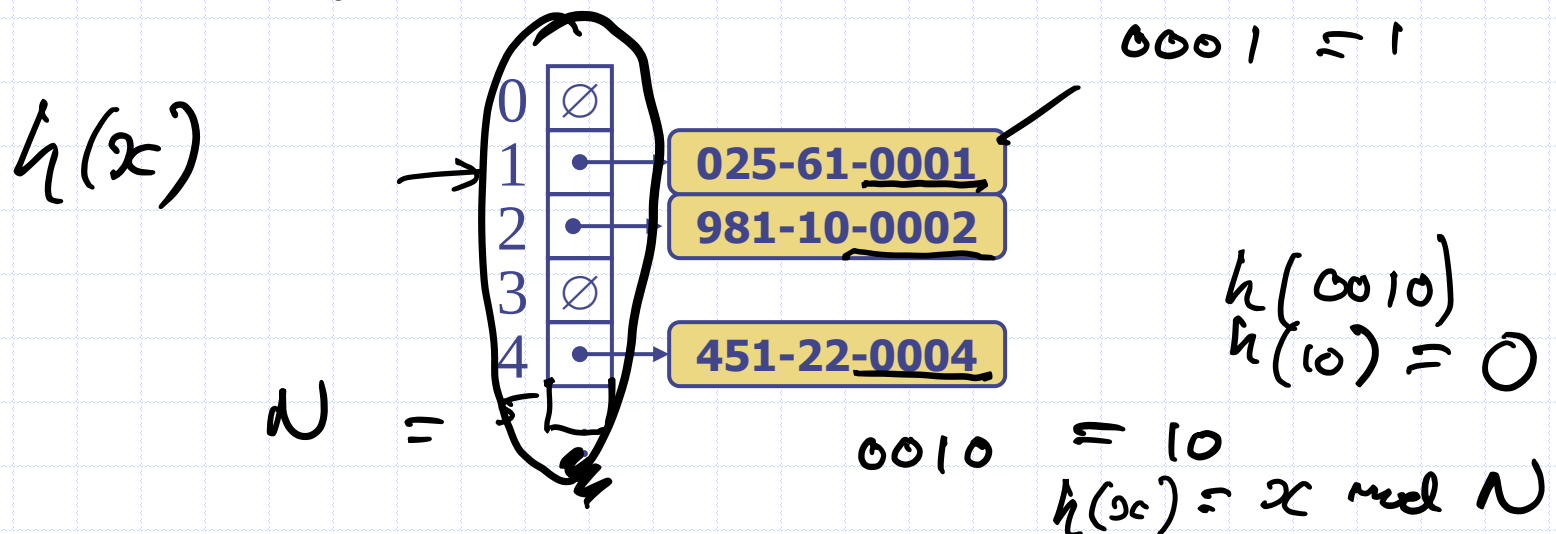
- That is, we *must* have collisions

$$\exists a, b : h(a) = h(b)$$

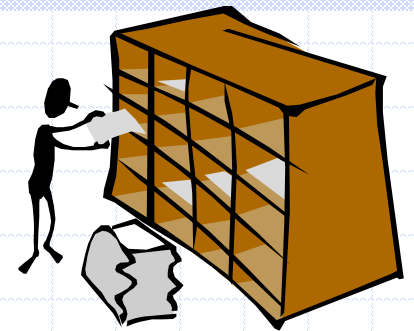
$$h(\text{"hello"}) = h(\text{"Berry"})$$

More General Kinds of Keys

- What should we do if our keys are not integers in the range from 0 to $N - 1$?
 - use a **hash function** to map general keys to corresponding indices in a table
 - e.g. last four digits of a Tax File Number



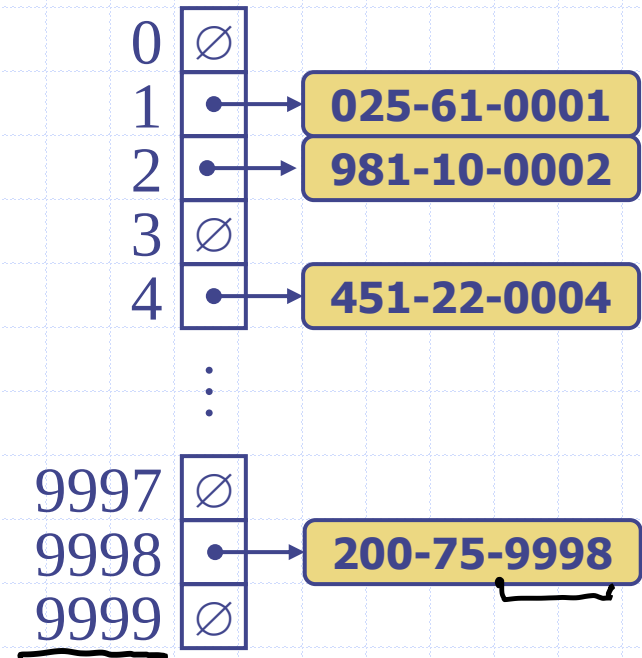
Hash Functions and Hash Tables



- Hash function h maps keys of a given type to integers in a fixed interval $[0, N - 1]$
 - $h(x) = x \bmod N$ – hash function for integer keys
 - Integer $h(x)$ is called the hash value of key x
 - Hash table for a given key type consists of
 - hash function h
 - bucket array (called table) of size N
- Map implemented with a hash table
 - goal is to store item $(\underline{k}, \underline{o})$ at index $i = h(k)$

Example

- Hash table for a map storing entries as (TFN, Name)
→ where TFN is a nine-digit positive integer
- Hash table uses an array of size $N = 10,000$ and the hash function
 - $h(x) = \text{last four digits of } x$



Hash Functions



- Usually specified as the composition of two functions

- **Hash code**

- h_1 : keys $\rightarrow$ integers

- **Compression function**

- h_2 : integers $\rightarrow [0, N - 1]$

- Hash code is applied first
- Compression function is applied on the result
 - i.e. $h(x) = h_2(h_1(x))$

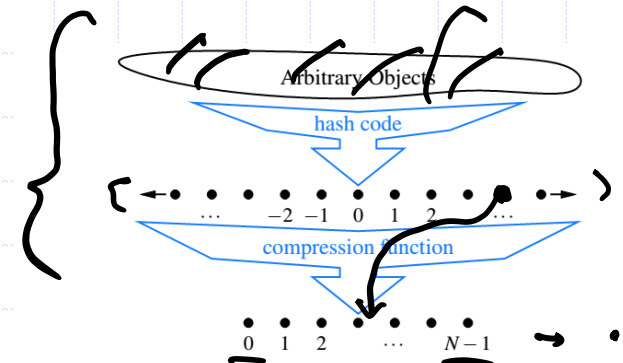
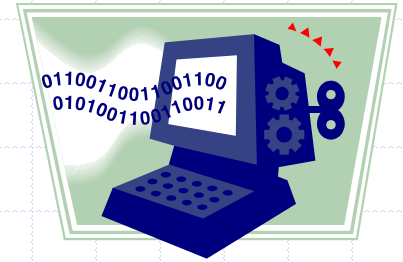


Figure 10.5: Two parts of a hash function: a hash code and a compression function.

Hash Codes



- **Component Sum**

- Partition the bits of the key into components of fixed length
 - ◆ e.g. 16 or 32 bits
- Sum the components

python
`ord('c')`

Hash Codes & Strings

- Consider component sum of characters in a string

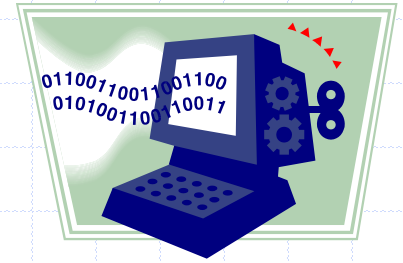
ASCII: $h = 104$

- "the"

- ♦ $h = \underbrace{(\text{int}) t + (\text{int}) h + (\text{int}) e}$

- Collisions may be common

- the = teh = eht = het = eth



Hash Codes (cont.)

■ Polynomial Accumulation

- Partition the bits of the key into a sequence of components of fixed length (e.g. 8, 16 or 32 bits)

◆ $a_0 a_1 \dots a_{n-1}$

- Evaluate the polynomial

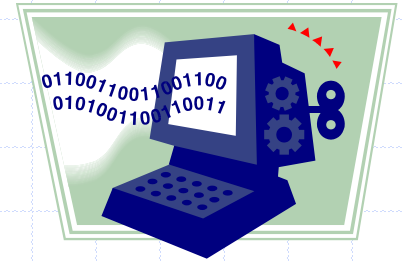
$$p(z) = \underline{a_0} + \underline{a_1 z} + \underline{a_2 z^2} + \dots + \underline{a_{n-1} z^{n-1}}$$

at a fixed value z , ignoring overflows

Empirically, $z = \{33, 37, 39, 41\}$ are reasonable.

This is a type of “rolling” hash function – each step depends on the value of the previous step

Hash Codes (cont.)



- What is the intuition?
 - We are trying to spread out the influence of each “piece” of the key on the final hash code
 - This means the hash code won't depend too much on any individual “piece”
 - By increasing powers on each successive piece, we ensure significant bits are set/utilized

Note: Here the “pieces” are those 8/16/32-bit quantities from the original key, $a_0 a_1 \dots a_{n-1}$

≈ 32 : 00
BFS

hello: $h \rightarrow 104$
 $e \rightarrow 101$
 $l \rightarrow 108$
 $o \rightarrow 111$ }

$h: 00000 \rightarrow \dots \rightarrow 01101000$

1. Baru - - - - 1101100

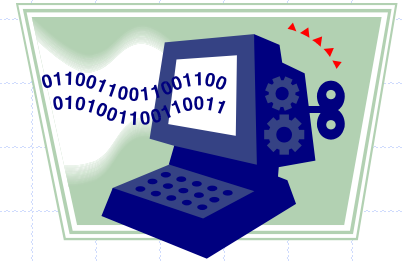
h e f l t o

$$102 + 101 + 108 + 108 + 111 = ?$$

၂၆၂၆၀ ၂၆၂၆၀ ၀၀၀ ၀၀၀၀၀၀၀၀၀၀ ၁၀၁.

$$hfc + 1 \dots \text{on } 11 \cdot 2^5$$

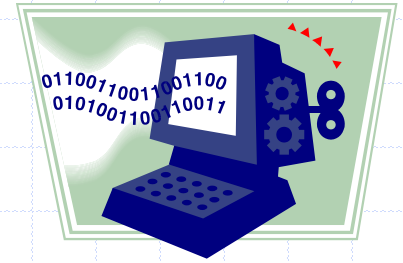
Hash Codes (cont.)



■ Cyclic Shift

- Variant of polynomial accumulation that replaces the multiplication with a bit shift
- Shift n bits from one side of bit pattern to the other side during the partial accumulation

<u>10001110</u>	original bit-string representation
01000111	1-bit cycle shift
10100011	2-bit
<u>11010001</u>	...



Hash Codes (cont.)

□ Cyclic Shift

```
def cyclic_hash(my_string):  
    mask = (1 << 32) - 1    # Max 32-bit int  
    h = 0                  # the running sum of the hash value  
    for character in my_string:  
        h = (h << 5 & mask) | (h >> 27) # 5 bit shift  
        h += ord(character) # the integer representation of the char  
    return h
```

Cyclic Shift Collisions Example

- ❑ Each row shows the result of shifting a different number of bits
- ❑ Shows collisions for 230,000+ English words
- ❑ Total – total number of collisions
- ❑ Max – largest number of collisions that occurred for a hash code
- ❑ Which code is best?
 - (For this data)

Shift	Collisions	
	Total	Max
0	234735	623
1	165076	43
2	38471	13
3	7174	5
4	1379	3
5	190	3
6	502	2
7	560	2
8	5546	4
9	393	3
10	5194	5
11	11559	5
12	822	2
13	900	4
14	2001	4
15	19251	8
16	211781	37



Compression Functions

□ Division

- $h_2(y) = y \bmod N$

- size N of the hash table should be prime

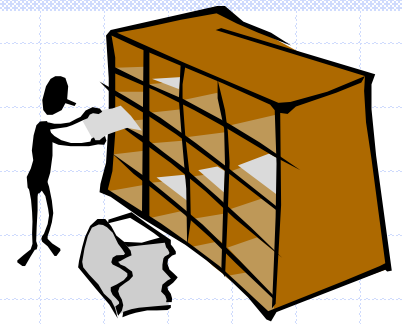
- ◆ reason has to do with number theory and is beyond the scope of this course
- ◆ Basic idea: Reduce the number of common factors between hash values and N
- ◆ Results in more "spread" across available buckets



Compression Functions

□ Multiply, Add and Divide (MAD)

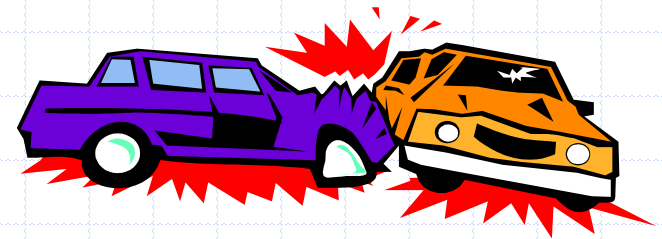
- $h_2(y) = ((ay + b) \bmod p) \bmod N$
- p is a prime number $> N$
- a and b are random integers in the range $[0, p-1]$



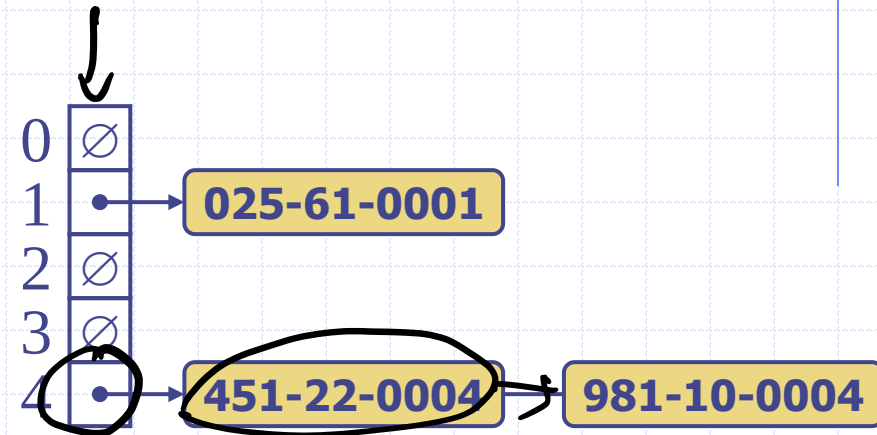
Recall That

- Hash table for a given key type consists of
 - hash function h that maps keys of a given type to integers in a fixed interval $[0, N-1]$
 - array (called table) of size N
- Goal is to store item (k, o) at index $i = \underline{h(k)}$
 - might get collisions
- Collisions occur when different elements are mapped to the same cell

Collision Handling



- ❑ Collisions occur when different elements are mapped to the same cell



- ❑ **Separate Chaining**
 - each cell in the table points to a linked list of entries that map there
 - simple, but requires additional memory outside the table

Map with Separate Chaining

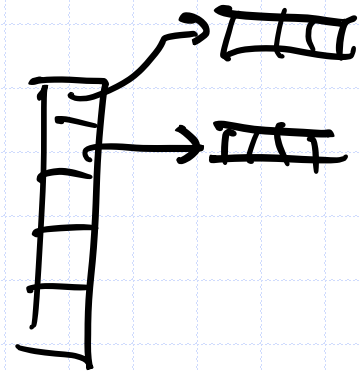
- Delegate operations to a list-based map at each cell

Algorithm *get*(k): # *get* is find
return $A[h(k)].get(k)$

Linked list → get

Algorithm *put*(k, v): # *put* is insert
t = $A[h(k)].put(k, v)$
if t = **null** **then** {k is a new key}
 n = n + 1
return t

Algorithm *remove*(k):
t = $A[h(k)].remove(k)$
if t ≠ **null** **then** {k was found}
 n = n - 1
return t

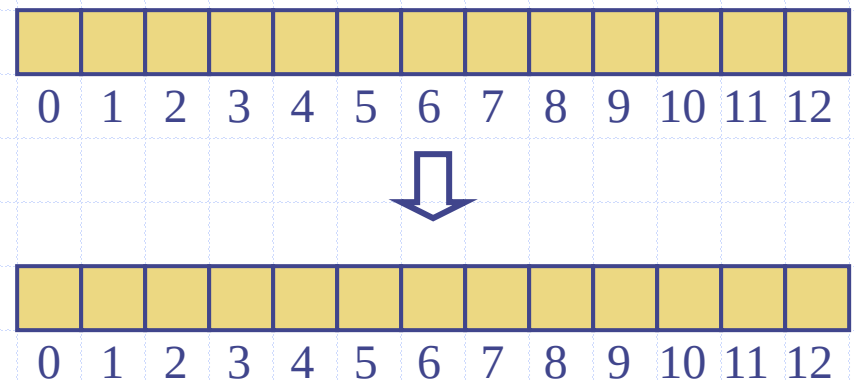


Linear Probing

- **Open addressing:** colliding item is placed in a different cell of the table
- **Linear probing:** handles collisions by placing the colliding item in the next (circularly) available table cell
- Each table cell inspected is referred to as a “probe”
- Colliding items lump together
 - causing future collisions to cause a longer sequence of probes

Example

- $h(x) = x \bmod 13$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

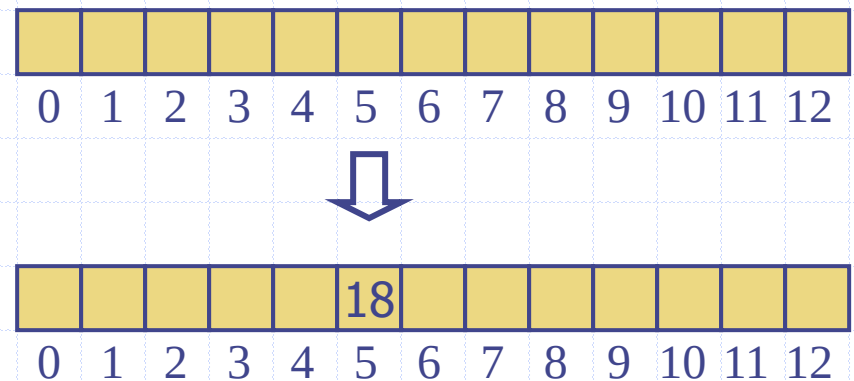


Linear Probing

- ❑ **Open addressing:** colliding item is placed in a different cell of the table
- ❑ **Linear probing:** handles collisions by placing the colliding item in the next (circularly) available table cell
- ❑ Each table cell inspected is referred to as a “probe”
- ❑ Colliding items lump together
 - causing future collisions to cause a longer sequence of probes

❑ Example

- $h(x) = x \bmod 13$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

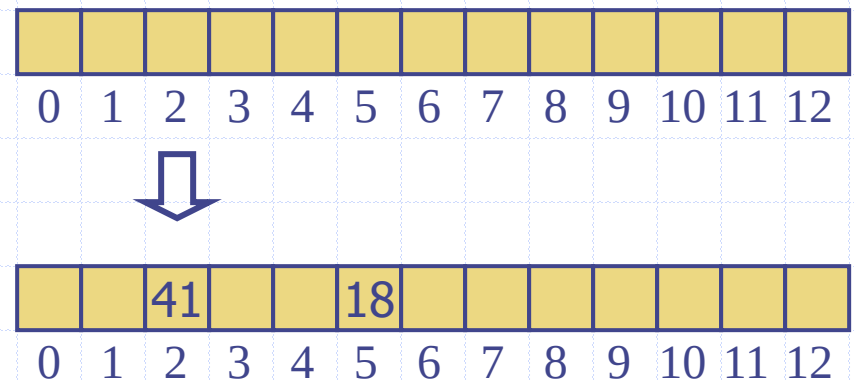


Linear Probing

- ❑ **Open addressing:** colliding item is placed in a different cell of the table
- ❑ **Linear probing:** handles collisions by placing the colliding item in the next (circularly) available table cell
- ❑ Each table cell inspected is referred to as a “probe”
- ❑ Colliding items lump together
 - causing future collisions to cause a longer sequence of probes

❑ Example

- $h(x) = x \bmod 13$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order



Linear Probing

- ❑ **Open addressing:** colliding item is placed in a different cell of the table
- ❑ **Linear probing:** handles collisions by placing the colliding item in the next (circularly) available table cell
- ❑ Each table cell inspected is referred to as a “probe”
- ❑ Colliding items lump together
 - causing future collisions to cause a longer sequence of probes

❑ Example

- $h(x) = x \bmod 13$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

0	1	2	3	4	5	6	7	8	9	10	11	12

↓

		41			18				22			
0	1	2	3	4	5	6	7	8	9	10	11	12

Linear Probing

- ❑ **Open addressing:** colliding item is placed in a different cell of the table
- ❑ **Linear probing:** handles collisions by placing the colliding item in the next (circularly) available table cell
- ❑ Each table cell inspected is referred to as a “probe”
- ❑ Colliding items lump together
 - causing future collisions to cause a longer sequence of probes

❑ Example

- $h(x) = x \bmod 13$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

0	1	2	3	4	5	6	7	8	9	10	11	12

↓

		41			18	44			22			
0	1	2	3	4	5	6	7	8	9	10	11	12

Linear Probing

- ❑ **Open addressing:** colliding item is placed in a different cell of the table
- ❑ **Linear probing:** handles collisions by placing the colliding item in the next (circularly) available table cell
- ❑ Each table cell inspected is referred to as a “probe”
- ❑ Colliding items lump together
 - causing future collisions to cause a longer sequence of probes

❑ Example

- $h(x) = x \bmod 13$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

0	1	2	3	4	5	6	7	8	9	10	11	12

↓

		41			18	44	59		22			
0	1	2	3	4	5	6	7	8	9	10	11	12

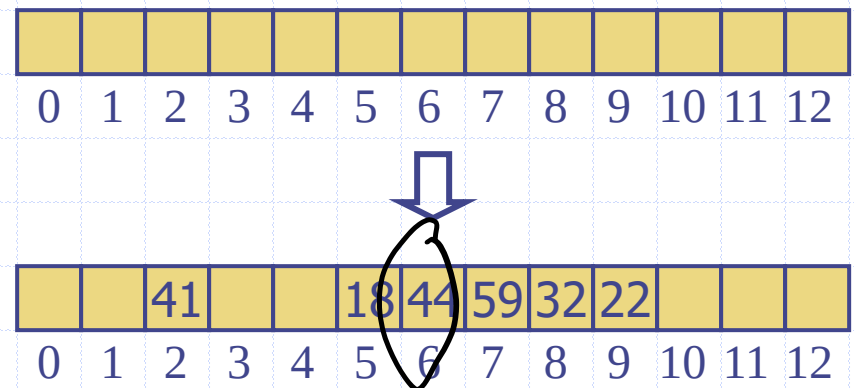
Linear Probing

- ❑ **Open addressing:** colliding item is placed in a different cell of the table
- ❑ **Linear probing:** handles collisions by placing the colliding item in the next (circularly) available table cell
- ❑ Each table cell inspected is referred to as a “probe”
- ❑ Colliding items lump together
 - causing future collisions to cause a longer sequence of probes

❑ Example

- $h(x) = x \bmod 13$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

0	1	2	3	4	5	6	7	8	9	10	11	12
		41			18	44	59	32	22			
0	1	2	3	4	5	6	7	8	9	10	11	12



Linear Probing

- ❑ **Open addressing:** colliding item is placed in a different cell of the table
- ❑ **Linear probing:** handles collisions by placing the colliding item in the next (circularly) available table cell
- ❑ Each table cell inspected is referred to as a “probe”
- ❑ Colliding items lump together
 - causing future collisions to cause a longer sequence of probes

❑ Example

- $h(x) = x \bmod 13$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

0	1	2	3	4	5	6	7	8	9	10	11	12
↓												
		41			18	44	59	32	22	31		
0	1	2	3	4	5	6	7	8	9	10	11	12

Diagram illustrating the insertion of keys into a hash table using linear probing. The initial state shows an empty table. The keys 18, 41, 22, 44, 59, 32, 31, and 73 are inserted in order. The final state shows the keys 41, 18, 44, 59, 32, 22, and 31 stored in the table, with 73 not shown. The keys 18, 44, 59, 32, 22, and 31 are grouped together, indicating a cluster of colliding items.

Linear Probing

- ❑ **Open addressing:** colliding item is placed in a different cell of the table
- ❑ **Linear probing:** handles collisions by placing the colliding item in the next (circularly) available table cell
- ❑ Each table cell inspected is referred to as a “probe”
- ❑ Colliding items lump together
 - causing future collisions to cause a longer sequence of probes

❑ Example

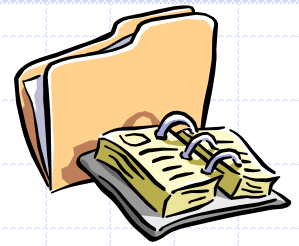
- $h(x) = x \bmod 13$
- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

0	1	2	3	4	5	6	7	8	9	10	11	12

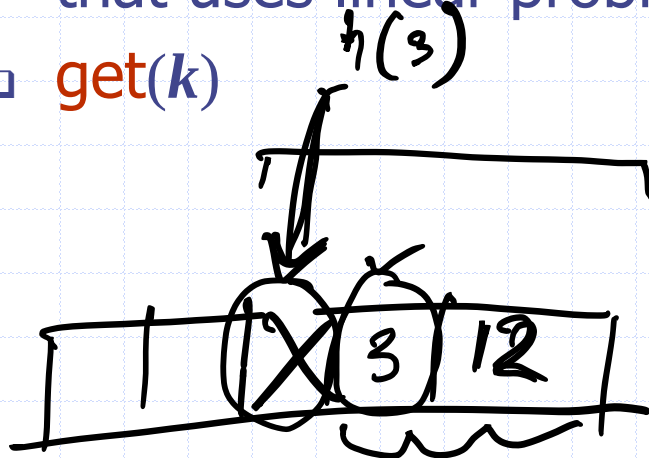
↓

		41			18	44	59	32	22	31	73	
0	1	2	3	4	5	6	7	8	9	10	11	12

Search with Linear Probing



- Consider a hash table A that uses linear probing
- $\text{get}(k)$



Algorithm $\text{get}(k)$:

$i \leftarrow h(k)$

$p \leftarrow 0$

repeat

$c \leftarrow A[i]$

if $c = \emptyset$

return *null*

else if $c.\text{getKey}() = k$

return $c.\text{getValue}()$

else

$i \leftarrow (i + 1) \bmod N$

$p \leftarrow p + 1$

until $p = N$

return *null*

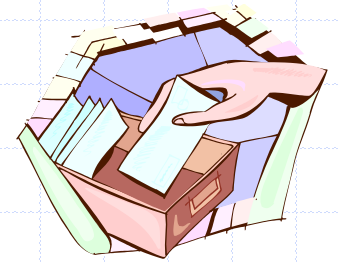
Updates with Linear Probing (2)

□ $\text{put}(k, o)$

- throw an exception if the table is full
- start at cell $h(k)$
- probe consecutive cells until one of the following occurs
 - ◆ cell i is found that is either empty, or
 - ◆ N cells have been unsuccessfully probed
- store (k, o) in cell i

Quadratic Probing

- Probe index calculated by quadratic function
 - $(h(k) + f(i)) \bmod N$
 - ♦ where $f(i) = i^2$, for $i = 0, 1, 2, \dots$
- Guaranteed to find an empty slot if
 - table size is prime
 - table is less than half full
 - This is because the quadratic probing sequence must visit at least half of the table indexes before repeating if the size of the table is prime



Double Hashing

- Secondary hash function $d(k)$ handles collisions by placing an item in the first available cell of the series $(h(k) + jd(k)) \bmod N$ for $j = 1, 2, \dots, N - 1$

$$x = h(k)$$
$$y = d(k)$$

Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - k \bmod 7$
 - $(h(k) + jd(k)) \bmod 13$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



					18							
0	1	2	3	4	5	6	7	8	9	10	11	12

Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - k \bmod 7$
 - $(h(k) + jd(k)) \bmod 13$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



		41			18							
0	1	2	3	4	5	6	7	8	9	10	11	12

Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - k \bmod 7$
 - $(h(k) + jd(k)) \bmod 13$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



		41			18				22			
0	1	2	3	4	5	6	7	8	9	10	11	12

Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - k \bmod 7$
 - $(h(k) + jd(k)) \bmod 13$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



		41			18				22			
0	1	2	3	4	5	6	7	8	9	10	11	12



Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - k \bmod 7$
 - $(h(k) + jd(k)) \bmod 13$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



		41			18				22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12

Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - k \bmod 7$
 - $(h(k) + jd(k)) \bmod 13$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



		41			18		59		22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12

Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - k \bmod 7$
 - $(h(k) + jd(k)) \bmod 13$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



		41			18	32	59		22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12

Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - k \bmod 7$
 - $(h(k) + jd(k)) \bmod 13$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



		41			18	32	59		22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12

Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - k \bmod 7$
 - $(h(k) + j \cdot d(k)) \bmod 13$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



		41			18	32	59		22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12

1

Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - k \bmod 7$
 - $(h(k) + jd(k)) \bmod 13$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



31		41			18	32	59		22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12

Example of Double Hashing

- Consider a hash table storing integer keys that handles collision with double hashing

- $N = 13$
 - $h(k) = k \bmod 13$
 - $d(k) = 7 - k \bmod 7$
 - $(h(k) + jd(k)) \bmod 13$

- Insert keys 18, 41, 22, 44, 59, 32, 31, 73, in this order

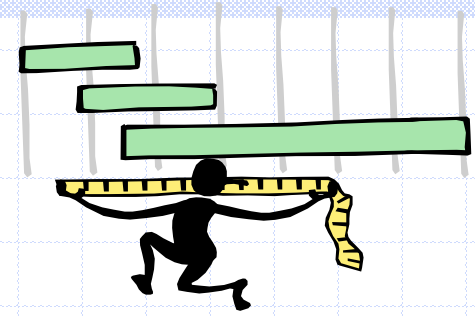
k	$h(k)$	$d(k)$	Probes	
18	5	3	5	
41	2	1	2	
22	9	6	9	
44	5	5	5	10
59	7	4	7	
32	6	3	6	
31	5	4	5	9 0
73	8	4	8	

0	1	2	3	4	5	6	7	8	9	10	11	12



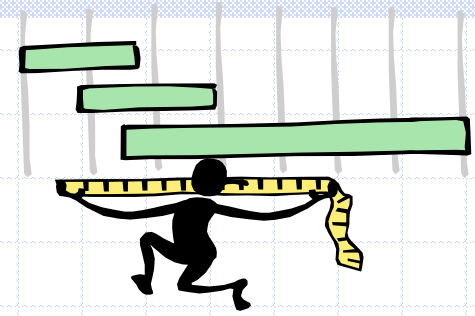
31		41			18	32	59	73	22	44		
0	1	2	3	4	5	6	7	8	9	10	11	12

Hashing Performance



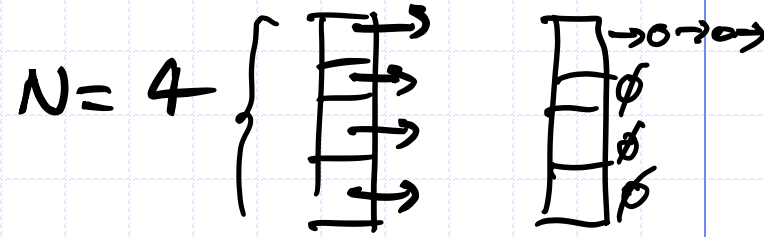
- $O(1)$ – expected running time of all Map operations
- $O(n)$ – worst case for searches, insertions and removals
 - occurs when all keys inserted into the map collide

Hashing Performance



- Load factor $\alpha = \frac{n}{N}$ affects performance of a hash table
- Expected number of probes for an insertion with open addressing is
 - $1 \div (1 - \alpha)$
- So what might be a reasonable load factor?

Load Factors



- Chaining: $\alpha = 0.8$ to 1.0 acceptable
 - Low space overhead; a good hash function will give us on average α elements per linked list
 - Note that load can be arbitrarily large (chains approach infinite length as α approaches infinity)
- Probing: Depends on the type, but typically $\alpha < 2/3$ is desired
 - Python's own open addressing uses $2/3$
 - State-of-art mechanisms can do well even with $\alpha > 0.9$
 - Can $\alpha > 1.0$?

Swiss Table

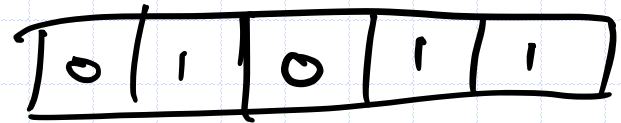
Fun with Hashing

- We can use hashing to implement structures other than maps
 - Bloom Filters – fast member testing
- Want collision free hashes?
 - Perfect hash functions
- Want less probing (and, worst-case constant time lookups)?
 - Cuckoo hashing

Bloom Filters

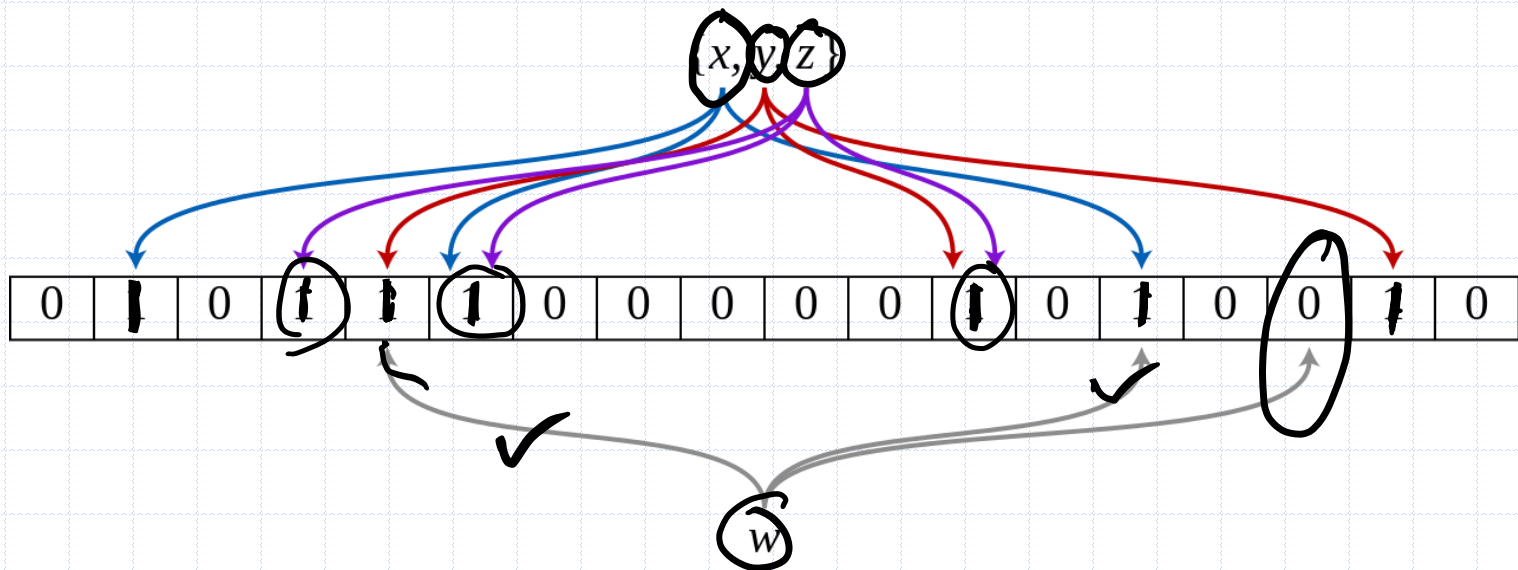
- Sometimes we might want to answer questions like “is the key k in my set S or not” (think duplication detection for example)
- **Idea:** We will build a structure that can efficiently answer this question with a caveat
 - **If the answer is “no”** then k is **not** in S
 - **If the answer is “yes”** then k **might be** in S
 - That is, false positives are possible, but false negatives are not

Bloom Filters



- Define an empty m element bitvector
 - Proportional to the number of keys to add, n
- Define z unique hash functions
 - z is usually a small constant
- To add a key, pass it to each of the z hash functions
 - Each switches on a bit
- To query a key, do the same as "add" and see if all bits are on

Bloom Filters



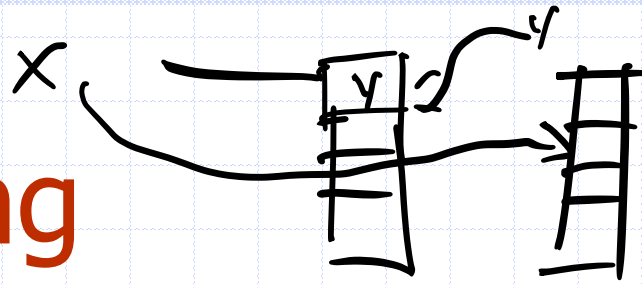
<https://doris.apache.org/docs/dev/data-table/index/bloomfilter/>

Perfect Hashing

$[0, 1000]$

- **Idea:** Given our set of keys S , we do some “offline work” to find a hash function that gives no collisions
- Then, we know that we can always get constant-time access to those entries!
- Example: $h(k) = k \bmod 10$ is perfect for $S = \{\underline{10}, \underline{21}, \underline{32}, \underline{43}, \underline{54}, \underline{65}, \underline{76}, \underline{87}, \underline{98}\}$
- Knowing about keys ahead of time is very useful for many applications
 - What if they are integers from 0 to n ?

Cuckoo Hashing



- **Idea:** Store two hash tables with a unique hash function for each

- **Put(k):** Try the first table; if occupied, throw the entry with key k' out and enter k here; Now try to put k' into the second table... Continue repeatedly until a pre-set number of these repeats occur (or a cycle is found) – if so, rehash both tables with a new set of hash functions!!

- **Get(k):** It's either at $h_1(k)$ or $h_2(k)$...

- ◆ Guaranteed constant-time access

Cuckoo Hashing

- Invented by Rasmus Pagh (a grad student in 2001)
- **Wildly cool? You bet!**
 - Insertions (even though you might end up with a big song and dance with rehashing keys) have been proven to be *expected constant time*
 - ♦ So long as the load factor is below 50%
 - Finds/Deletes are constant time
 - Wow!!!

Cuckoo Hashing in practice...

- A common theme in my “side stories”
 - The cache will kill us
 - Why? Cuckoo hashing will involve looking at two indexes, and those indexes are likely to be far apart
 - ◆ 2 cache misses per access
 - Linear probing, on the other hand?
 - ◆ Hash to a location, and then walk forwards over contiguous memory! Likely just one cache miss per access... And hence, likely faster in practice.



Hashing *Gone Wild*

- Rule of thumb: Never try to make your own hash function!
 - It will probably suck!!!!!!

Hashing *Gone Wild*

- Rule of thumb: Never try to make your own hash function!
 - It will probably suck!!!!!!
- In assignment 2, you get to make your own hash function!
 - It will probably suck!!!!!!
 - But you will hopefully learn something from your experience...

Another note on hash functions

- Hash functions for data structures
 - Need to be fast!
 - Consider trade-off between time to compute the hash function vs handling collisions...

- Checksums
 - Used for verifying data (if data changes, so too does checksum, usually by large magnitude)
 - Cryptographic hashes
 - "One way" functions, secure to malicious behaviour.
 - Easy to go $\text{key} \rightarrow \text{hash}$, "impossible" $\text{hash} \rightarrow \text{key}$
- "Barry is a legend" $\Rightarrow \checkmark$

NOTE

Monday Sep 4, 2023

Remainder of this content has been shifted into the week 7 lecture. Please see the Week 7 slides BST's and AVL trees, plus more.

Further Reading

- Data Structures and Algorithms in Python
 - Chapter 10.2-10.3 ✓ and 11.1-11.3
- Introduction to Algorithms
 - Chapter 11 ✓
 - Chapter 12

Trees, week 7